

CS1010E: Programming Methodology

Assignment 4: Map, Filter, Search, Sort, OOP

Instructions

Submission Instructions

Read ALL instructions carefully.

1. Submission Content:

- Each question will be given a template file (e.g., `aN_qM_template.py`).
 - You should work on your file locally before submitting it to Coursemology.
- Comment out all the given **test cases** and **import** statements before copying the content of the entire file to Coursemology, *replacing* any content on Coursemology: – On template file: CTRL + A followed by CTRL + C (i.e., *copy all*).
 - On Coursemology: CTRL + A followed by CTRL + V (i.e., *replace all*).
- If any implementation is given, you are not allowed to change them.
- You are not allowed to *add, remove, or modify* any parameters from functions.
- You are not allowed to change the *name* of the given function.
- Unless otherwise stated, you are not allowed to add/change **import**.
- This assignment contains 6 Questions, where **Question 4 and Question 5** convey **1 CA mark** each.

2. Finalizing Submissions:

- You are responsible for finalizing your submission.
- Late submission is **not allowed**
- You should not submit at the last minute since Coursemology is not designed to handle a large workload.
- If you see that Coursemology is *under maintenance*, it is due to the large workload and not actual maintenance. No extension will be given.

3. Testing:

- Do not call the required functions when you copy the code to Coursemology as it will be called automatically during testing.
- Your function should **return** the result instead of **print**.
- Test cases are called in succession (i.e., *no reverting of global state*).
- Passing all test cases (*public or private*) does not guarantee full marks as we will do a manual evaluation on other aspects such as code quality.
- You may assume that the code will only be tested on valid inputs that follow the assumption. Do NOT put any checks for inputs with respect to the assumptions, UNLESS it is directly asked in the Question.

4. Good Programming Practice:

- Codes that are redundant or too long may be penalized.
- Restrictions must not be violated.
- Each test case (*public or private*) should complete within a reasonable amount of time typically within 1s to 2s.

Failure to follow any of the instructions above will result in a deduction of your marks.

Files

- Assignment04.pdf
- A4_q1_template.py
- A4_q2_template.py
- A4_q3_template.py
- A4_q4_template.py
- A4_q5_template.py

Coursemology

- Assignment 4: Map, Filter, Search, Sort, OOP

**** Attention: This assignment contains 5 Questions, where
Question 4 and Question 5 convey 1 CA mark each.**

Deadline: 21-4-2023, 23:59

Question 1: Hash Keys (S&S)

Develop a program to get any input string and return a tuple of 2 hash keys computed for that string.

`(hk1 , hk2) = hash_keys(instr)`

```
def hash_keys( instr ): pass
    return (hk1,hk2,)
```

The first hash key would be computed by XORING all the characters of the input string *instr*. You need to find a way to get the code (ASCII or UNICODE) of each character.

The second hash key would be computed by multiplying characters of *instr* by consecutive powers of 2, adding the results together, and computing the remainder of its division by 1024.

Example:

```
instr = 'abcd' hk1 = a xor b xor c xor d hk2 =
(a*2**0 + b*2**1 + c*2**2 + d*2**3) % 1024
```

Points:

- **You need to find:** a way to access the characters of a string, a function that returns the ASCII/Unicode code of a character, and the XOR operator.
- Check the test cases 1 to 3, ('a', 'aA', 'Aa'). Why the hash keys are like that? Add your answer as a comment at the end of your code before submitting.

Test Cases:

```
>>> hash_keys( 'a' )
(97, 97)
>>> hash_keys( 'aA' )
(32, 227)
>>> hash_keys( 'Aa' )
(32, 259)
# ❶ Why? What do you think? Add your answers in
comments >>> hash_keys( 'hello world' )
(32, 114)
>>> hash_keys( 'egg & ham Python Mython' )
(58, 431)
>>> hash_keys( 'Amirhasan' ) (66, 47)
```

Question 2: A Special Hash Key and Nonce (S&S)

What if we would like to have a special hash key for any given input string? For instance, a hash key that always has got 000 at the end?

How we can do that when the input string is there and the hash function is fixed? Well, the solution is having a **nonce** string that we can change and concat with the main input string. Then we give this new string [instr + nonce] to the hash function and check the resulted hash key. If it's got that condition (e.g. having 000 at the end), we will stop. Otherwise, we change the nonce, concat the new nonce with the input string, and continue.

```
def my_nonce_hash( inp_str, hshfun ):    pass
return (nonce , hk)
```

Where `inp_str` is the input string, and `hshfun` is the hash function used. Nonce is the modified output string (the result of the concatenation of nonce + `inp_str`, and `hk` is the output hash code with '000' at the end (least significant digits). You may try one of the hash functions that you've already developed in Question 1 as the `hshfun`. If it doesn't work well you may modify that, e.g. by changing the base from 2 to 8 or 10.

Points:

- Your generated hash key should have '000' at the end, i.e. least significant digits.
- The nonce that you should use is a 4-character string.
- We need to change the least significant digits of your hash key. So, where to insert the nonce string, depends on your hash function. i.e., either [nonce + `inp_str`], or [`inpstr` + nonce] could be the option.
- This is applicable in blockchains and bitcoin.
- Output returned should be a tuple.

Algorithm:

1. Init your nonce, e.g. ' ' (4 spaces)
2. Concat it with `inp_str`
3. Call your hash function and compute the hash
4. If the LS digits are 000, return the results
5. Otherwise, change the nonce, and go to 2;

Examples: (Attention: your results could be different due to using a different hash function. `hash2` is the optional hash function that was employed here. You need to use yours.

```
>>> my_nonce_hash( 'avfbg', hash2)
Found ! string= " øE avfbg ", hash= 1964596000 after 8505 tries. >>> my_nonce_hash( 'aA', hash2)
Found ! string= " P aA ", hash= 2546000 after 49 tries.
>>> my_nonce_hash( 'egg & ham Python Mython', hash2 )
Found ! string= " H egg & ham Python Mython ", hash= 38008215345419646966045000 after 41 tries.
>>> my_nonce_hash( 'Amirhasan', hash2 )
Found ! string= " øW Amirhasan ", hash= 8530880814000 after 12537 tries.
```

Question 3: Football Club Class (OOP)

We are going to develop a class for football clubs, called `my_fc`. The **attributes** or data members will be:

- `Club_owner`
- `Coach`
- List of footballers and their price: `foot_lst`
- `Division`
- `Capital`

Methods/member functions of `my_fc`:

- `Print`: to print all the information of a team
- `Sell`: to sell a footballer, he should be removed from the list of footballers, his price should be added to the capital.
- `Buy`: to buy a footballer. He would be inserted to the list. His price should be subtracted from the capital. If **capital - price < \$1000**, buying is not possible.

Sub-class: The `B_Team` `club_owner` and `capital` would be inherited from the parent class. Other attributes would be declared independently. Extra attribute is `a_team`, which is the name of the club that this `B_team` belongs to.

The only method to be added is '`transfer`', to transfer a player to the `my_fc` without any money transfer.

Scenario:

- 1- Your team AA, has got 3 footballers, [AA1, AA3, AA9], each worths \$10000. It has got \$8000 capital too.
- 2- Team AB is the B-team of AA. It has got 2 footballers, [AB1, AB2], each worths \$5000.
- 3- The team AA would like to buy Ronaldo, that worth \$45000.
- 4- To do that, firstly footballers of AB should be transferred to AA, then all AA footballers should be sold, then Ronaldo will be bought.

After defining classes `my_fc`, `B_team`, in your main function, league:

- 1- Then define instances AA and AB with requested attributes (see the scenario above)
- 2- Execute the steps described in step 3 and step 4 of the scenario above.
- 3- Return the list of footballers of AA and its capital.

```
Class my_fc():  
    Pass  
  
Class B_team(my_fc):  
    Pass  
  
def league():  
    pass  
    return [ AA.foot_lst , AA.capital ]
```

Result: We expect to see

```
[ [['Ronaldo' , 45000]] , 3000 ]
```

Question 4:**2d Binary Matrices Class (OOP)****[1 CA Mark]**

We are going to develop a class for 2d binary matrices. We simply call that class 'bmatrix'. let's limit the class to only binary numerical matrices (elements are either 0 or 1).

```
class bmatrix:  
    pass
```

When an instance of the class bmatrix is defined (__init__), declaration of the number of rows and columns is obligatory. The 3rd parameter is the initial value of the matrix (See the examples and test cases below).

Attributes/ Data members:

- MTX: the binary matrix
- rows: number of rows
- cols: number of columns

Methods/ Member functions:

- in your constructor, __init__(), you need the fill parameter too. Fill is either 0 or 1 to initiate your matrix.
- print(self, spt): to print the matrix, nice and ordered, spt is the separator character which separates the columns. Default is space (' ').
- How to define a default parameter?
- add(self, N): to binary add to matrices of the same size. It must check the sizes. N is the other matrix. if N has got different size, an error message should be printed and self shouldn't be changed. By binary addition we mean a kind of modula-2 addition: $0 \leftarrow 0+0$, $1 \leftarrow 0+1$, $1 \leftarrow 1+0$, $0 \leftarrow 1+1$
- set_elem(self,r,c,stuff): set the self's element at [r][c] to stuff. Stuff only can be 0 or 1. Anything else means printing an error message and doing nothing.
- get_elem(self,r,c): returns the self[r][c]
- mul(self, N): element-wise multiplication of self and N, $self[i][j] *= N[i][j]$, size check is a must, just like the add method.
- b_or(self, N): bit or of self and N, element-wise. Size check is a must, just like the add method.
- list(self): to convert/ cast the object to an ordinary 2d list. Useful when you want to return the results for checking.

Test Cases

You need to add the test cases below to your module for testing and evaluation purposes:

```
#
# bmatrix class

#

class bmatrix:
    pass

#----- TEST CASES ----- def
bm_test1():
    aa      = bmatrix(3,4,0)  bb =
    bmatrix(3,4,1)
    aa.print(',')
    bb.print(' ')
    aa.add(bb)
    return aa.list()

#-----
def bm_test2():
    bb      = bmatrix(3,4,1)
    bb.set_elem(2,2,0)
    return bb.list()

#-----
def bm_test3():
    aa = bmatrix(5,5,1)
    bb = bmatrix(6,6,1)
    bb.set_elem(3,3,0)
    bb.set_elem(4,4,5.25)
    bb.mul(aa)  return
    bb.list()

#-----
def bm_test4():
    cc = bmatrix(5,5,0)
    dd = bmatrix(5,5,1)
    dd.set_elem(3,3,0)
    dd.set_elem(4,4,0)
    dd.b_or(cc)
    return dd.list()
```


Results: After running your program we expect those 4 test cases above to return:

```
>>> bm_test1()
0,0,0,0
0,0,0,0
0,0,0,0
1 1 1 1
1 1 1 1
1 1 1 1
[[1, 1, 1, 1], [1, 1, 1, 1], [1, 1, 1, 1]]
>>> bm_test2()
[[1, 1, 1, 1], [1, 1, 1, 1], [1, 1, 0, 1]]
>>> bm_test3()
Error ! wrong value
Error ! different sizes
[[1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1], [1, 1, 1, 0, 1, 1], [1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1]] >>> bm_test4()
[[1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1], [1, 1, 1, 0, 1, 1], [1, 1, 1, 1, 1, 0]]
```

=====

Question 5: Compute Double Integral

[1 CA Mark]

We are going to use Higher Order Functions to compute the double integral of any given 2variable function:

$$\int \int f(x,y) dx dy$$

You need to develop a Python program that be able to get any 2-variable function and 4 boundary parameters and compute the integral of that function inside the boundary asked.

In fact, as you know, this deterministic integral is the volume under the curve of the function f between $x \leq x \leq x_u$ and $y \leq y \leq y_u$, where x_l, x_u, y_l, y_u are 4 boundary parameters defined:

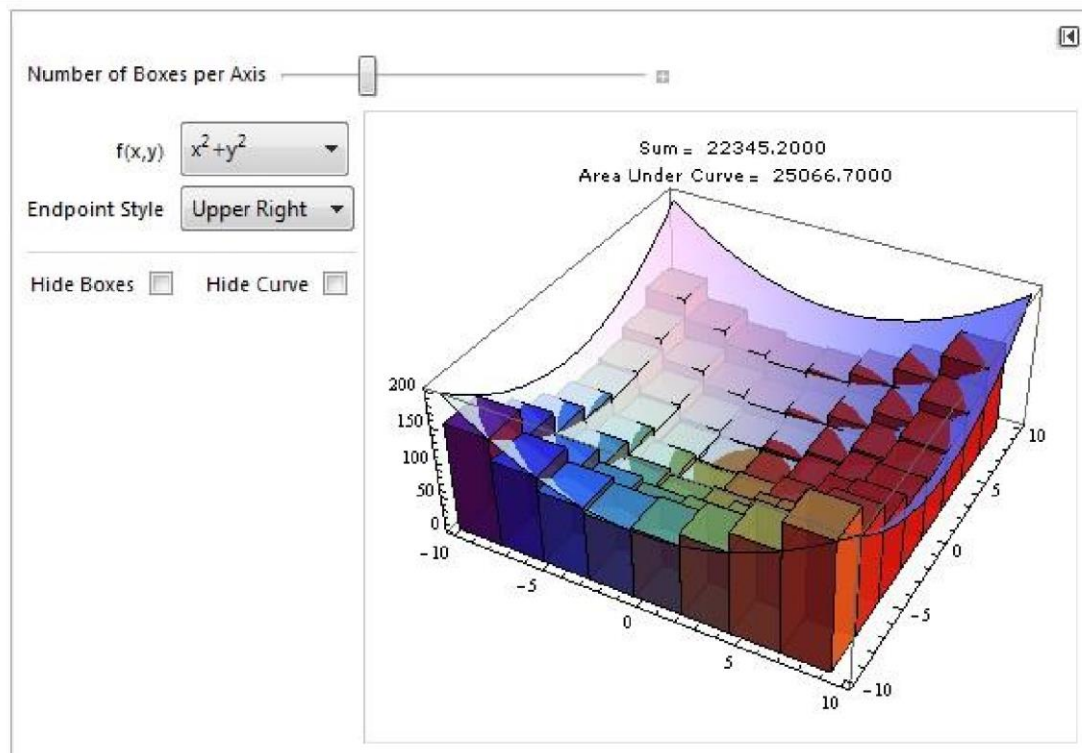
$$Z = my_2integral(f, xl, xu, yl, yu, dxdy)$$

```
def my_2integral(f, xl, xu, yl, yu, dxdy):  
    return z
```

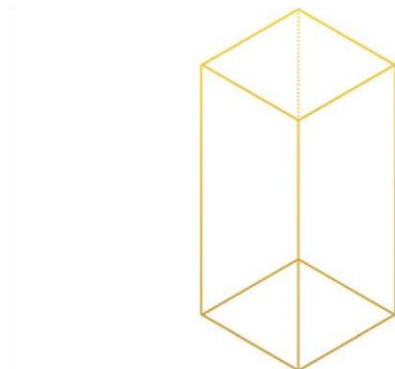
- f = 2 variable function $f(x,y)$, to be integrated.
- xl, xu = lower and upper band of the x variable.
- yl, yu = lower and upper band of the y variable.
- $dxdy$ = delta factor or length/width of the cuboids.
- z = computed integrals. It is a list of 2 integrals computed by different methods (cuboid and cut-cuboid, explained below. In fact, $z[0]$ = cuboid result, $z[1]$ = cut_cuboid result)
Obviously, to compute the volume under the curve, you may estimate that as a summation of volumes of many tiny cuboids or cut-cuboids (a cuboid with a slanted base). The dimensions of the base of that prisms/cuboids are $d_x d_y$. d_x and d_y are 2 small local parameters that you should declare in your functions.

We assume that $dx=dy$, and you must pass it as the last parameter to your function.

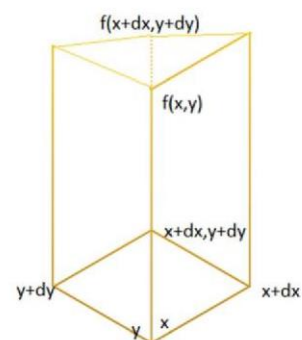
How to compute the volume of those cuboids? What about cut-cuboids? (don't forget that averaging always helps!)



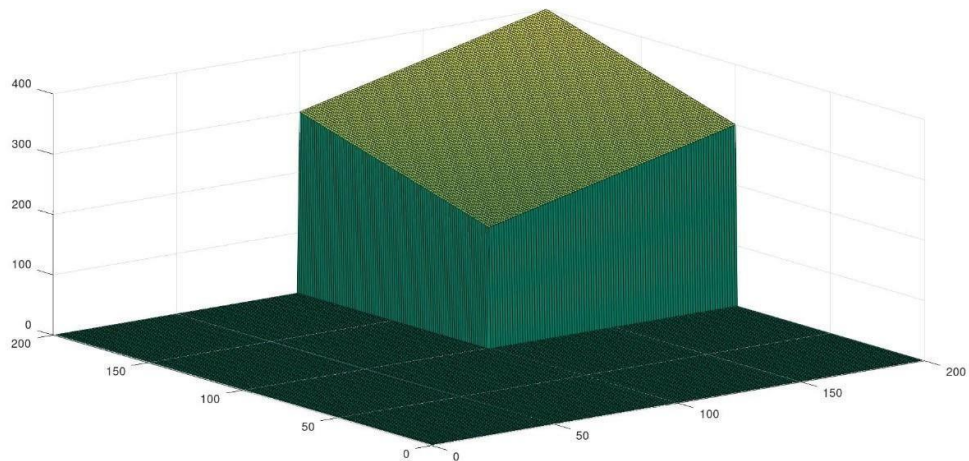
Reference: https://web.ma.utexas.edu/users/m408s/m408d/CurrentWeb/LM15_1-3.php



A cuboid



A cut-cuboid (one side is slanted), how to compute/estimate its volume? Don't forget averaging!



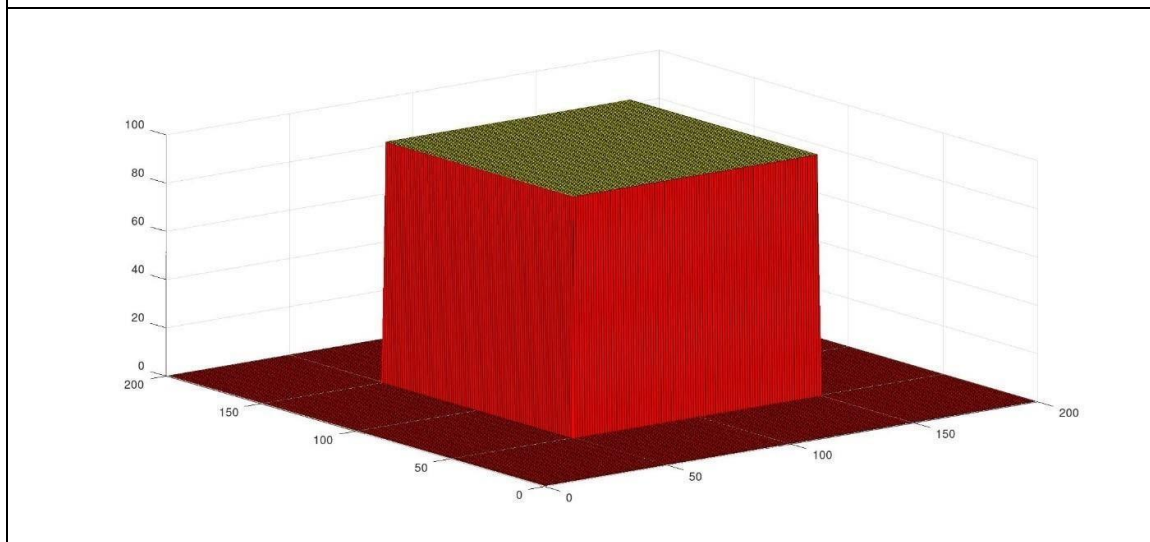
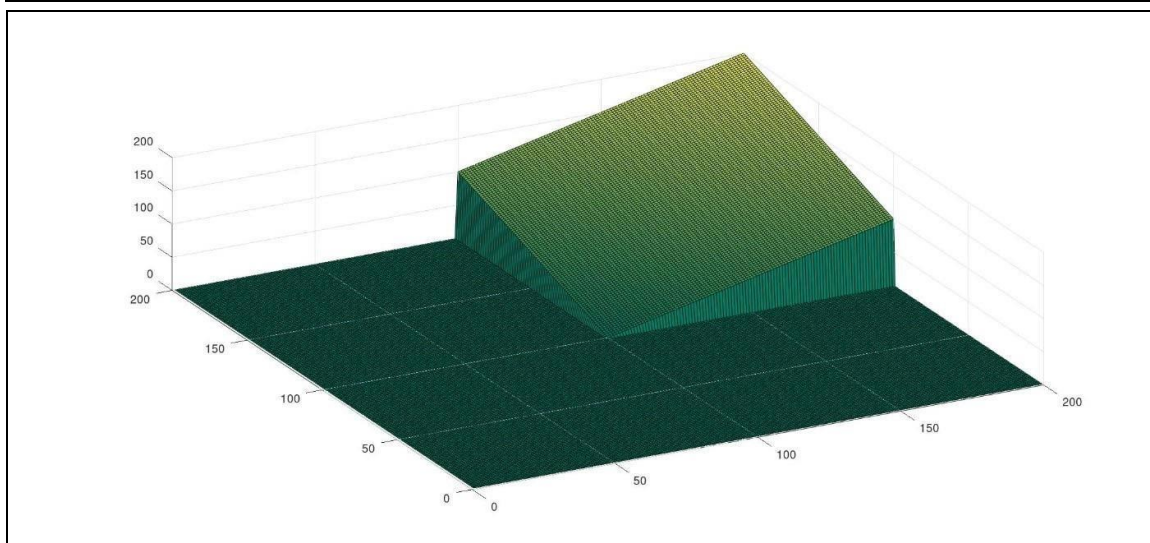
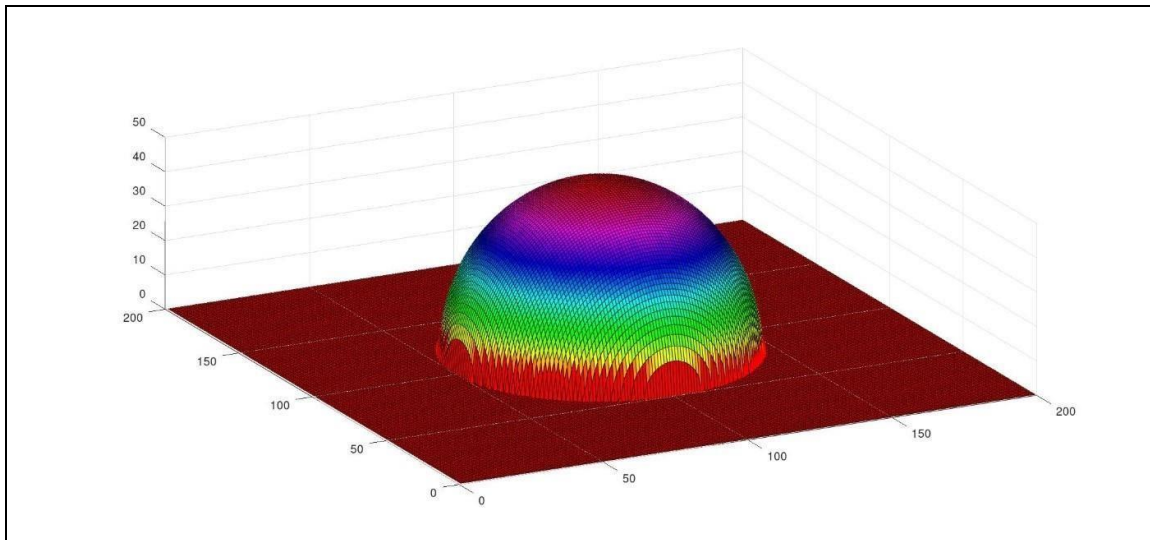
A cut-cuboid (one side is slanted), how to compute/estimate its volume?

You may try both cuboid and cut-cuboid and see the differences. Also, you may try different dx and dy and evaluate the results.

To test your program, you may try the double integrals below:

Function	Equation	Comments
2d Relu (use the name am_2d_relu for your function: am_2d_relu(x,y))	$f(x,y) = \begin{cases} 0, & x \text{ or } y < 0 \\ x + y, & x \text{ and } y \geq 0 \end{cases}$	What is the volume under the curve? Find it! For tests, assume: $-100 \leq x,y \leq 100$
Cube (function: cube(x,y))	$f(x,y) = \begin{cases} d, & x \text{ and } y \leq d \\ 0, & x \text{ or } y > d \end{cases}$	$v = d^3$ for tests, assume: $0 \leq x,y \leq 150$ $d = 50$
** Note: Hemisphere part is optional		
Hemisphere (function: hemisphere(x,y))	$f(x,y) = \begin{cases} \sqrt{r^2 - (x^2 + y^2)}, & x \text{ and } y \leq r \\ 0, & x \text{ or } y > r \end{cases}$	$v = \frac{2}{3}\pi r^3$ For tests, assume: $0 \leq x,y \leq 55$ $r = 20$

Images below help but they don't exactly match the specification.



Question

Develop a Python program, $Z = my_2integral(f, xl, xu, yl, yu, dx, dy)$. Using our program, you should be able to complete table like this:

Function	Parameters	Integral range	dx,dy	Method	Result
2d Relu	-	$-100 \leq x, y \leq 100$	0.1	Cuboid	
2d Relu	-	$-100 \leq x, y \leq 100$	0.1	Cut-cuboid	
2d Relu	-	$-100 \leq x, y \leq 100$	0.001	Cuboid	
2d Relu	-	$-100 \leq x, y \leq 100$	0.001	Cut-cuboid	
Real volume is around 1000000					
Cube	d=50	$0 \leq x, y \leq 150$	0.1	Cuboid	
Cube	d=50	$0 \leq x, y \leq 150$	0.1	Cut-cuboid	
Cube	d=50	$0 \leq x, y \leq 150$	0.01	Cuboid	
Cube	d=50	$0 \leq x, y \leq 150$	0.01	Cut-cuboid	
Real volume is 50^3 or 125000					
** Note: Hemisphere part is optional					
Hemisphere	r=20	$0 \leq x, y \leq 55$	0.1	Cuboid	
Hemisphere	r=20	$0 \leq x, y \leq 55$	0.1	Cut-cuboid	
Hemisphere	r=20	$0 \leq x, y \leq 55$	0.01	Cuboid	
Hemisphere	r=20	$0 \leq x, y \leq 55$	0.01	Cut-cuboid	
Real volume of that hemisphere (r=20) is 16755.15, half of that is 8377.57, quarter of that is 4188.79					

Result is the double integral of the function f mentioned, and the xl, xu, yl, yu parameters are declared in the integral range column. Also, dx, dy is defined there too.

Notes

- Forget the vertical/horizontal lines of the table, just make it readable and nice. They are comments anyway.
- Add some comments to your code and describe how the volume of cut-cuboids was computed.
- Mention the running time of your program too.
- You don't need to use any matrix to keep your function, unless you would like to plot it, which is optional.

- How to determine the method? Just implement both cuboid and cut-cuboid methods in your program, and return the results as a 1x2 list.
- About the optional hemisphere example, if the center of your hemisphere is in $\langle 0,0 \rangle$, your function computes a fraction (a half or a quarter) of the full hemisphere volume.
- **Finally**, you are supposed to add the completed table above as a comment to the end of your program. At least answers for 2d_Reluctance and Cube should be there. Forget the lines of the table, just contents do matter.

GOOD LUCK MY FRIENDS